

The encrypt side is symmetric GPG + ASCII armor + single-line base64. Decrypt is the exact reverse.

Decrypt

Bash

```
echo "$ENCRYPTED" | base64 --decode | gpg \  
  --decrypt \  
  --pinentry-mode=loopback \  
  --no-symkey-cache \  
  --quiet \  
  --no-greeting
```

That prints the original payload, e.g.:

text

```
RCLONE_CONFIG_SECRET_TYPE=crypt|RCLONE_CONFIG_MYSECRET_PASSWORD=sedd
```

Notes that match your encrypt flags

- `--pinentry-mode=loopback` so you can pass the passphrase non-interactively (`--passphrase-fd 0` or `GPG_TTY / gpg-agent` as you prefer).
- `--no-symkey-cache` so the session key is not cached after this run.
- Armor is already handled by `gpg --decrypt` after `base64 --decode` ; you do not need `-armor` on decrypt.
- Cipher / S2K / digest options are only needed on encrypt. Decrypt uses whatever is stored in the OpenPGP packet.

Passphrase on stdin (typical for scripts)

Bash

```
printf '%s\n' "$PASSPHRASE" | gpg \  
  --decrypt \  
  --pinentry-mode=loopback \  
  --passphrase-fd 0 \  
  --no-symkey-cache \  
  --quiet \  
  --no-greeting \  
< <(echo "$ENCRYPTED" | base64 --decode)
```

Or keep the pipe style you used on encrypt:

Bash

```
echo "$ENCRYPTED" | base64 --decode | \  
  gpg --decrypt --pinentry-mode=loopback --passphrase-fd 0 --no-symkey-  
cache --quiet --no-greeting \  
  <<< "$PASSPHRASE"
```

(The last form is a bit awkward; `--passphrase-fd 0` plus a here-string for the passphrase and the ciphertext on a different fd is cleaner.)

If you paste the actual decrypt command you use (or the encrypted blob + how you supply the passphrase) I can match it line-for-line